

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 1

Roll no: 54 PRN No: 12414148 Name of Student: Sanika Kulkarni

Subject Name: Deep Learning

Problem Statement:

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm:

1. Start the program.
2. Import TensorFlow, Keras, Matplotlib, and other required libraries.
3. Load the MNIST and Fashion MNIST dataset.
4. Divide the dataset into training and testing sets.
5. Normalize the image pixel values by scaling them to the range of 0 to 1 (divide by 255).
6. Display a few sample images from the dataset for visualization.
7. Build a sequential neural network model.
8. Add the input layer, one or more hidden layers, and the output layer.
9. Compile the model using the Adam optimizer and an appropriate loss function.
10. Train the model using the training dataset for a specified number of epochs
11. Evaluate the trained model using the testing dataset
12. Display the model's accuracy and loss values
13. Stop the program

Code:

1) Using MNIST dataset: Dataset of 70,000 grayscale images of handwritten digits (0–9) used for image classification.

```
import tensorflow as tf

import matplotlib.pyplot as plt

(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()

train_images = train_images / 255.0

test_images = test_images / 255.0

print(train_images.shape)

print(test_images.shape)

print(train_labels)

plt.imshow(train_images[11], cmap='gray')

plt.show()

my_model = tf.keras.models.Sequential()

my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))

my_model.add(tf.keras.layers.Dense(128, activation='relu'))

my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

my_model.fit(train_images, train_labels, epochs=3)

val_loss, val_acc = my_model.evaluate(test_images, test_labels)
```

```
print('Test accuracy: ', val_acc)

my_model.save('my_mnist_model.keras')
```

2) Using Fashion MNIST dataset: Dataset of 70,000 grayscale images of clothing items across 10 categories.

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models

from tensorflow.keras.utils import to_categorical

import matplotlib.pyplot as plt

import numpy as np

from tensorflow.keras.datasets import fashion_mnist

(trainX, trainy), (testX, testy) = fashion_mnist.load_data()

print(trainX.shape)

print(testX.shape)

print(trainy)

plt.imshow(trainX[6], cmap='gray')

plt.show()

my_model = tf.keras.models.Sequential()

my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))

my_model.add(tf.keras.layers.Dense(128, activation='relu'))

my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

```
my_model.fit(trainX, trainy, epochs=3)

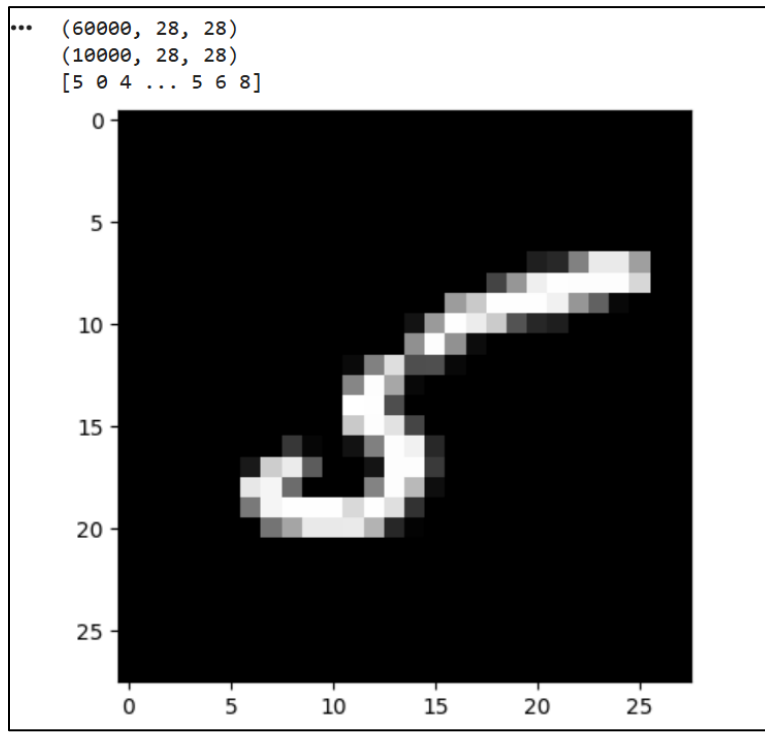
val_loss, val_acc = my_model.evaluate(testX, testy)

print('Test accuracy: ', val_acc)

my_model.save('my_fashion_mnist_model.keras')
```

Output:

First program:



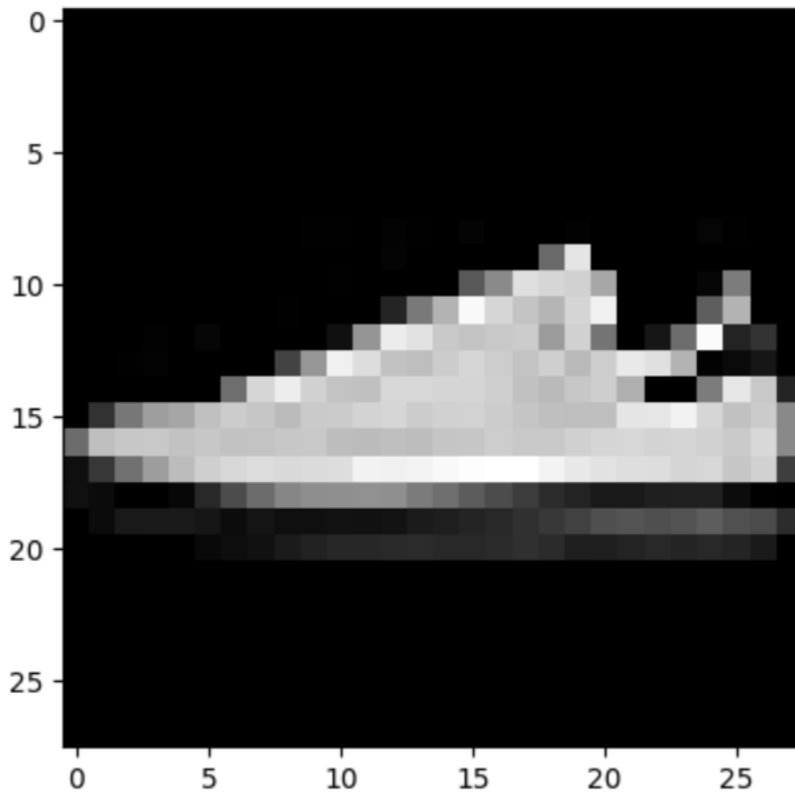
```
Epoch 1/3
1875/1875 ————— 5s 2ms/step - accuracy: 0.7253 - loss: 1.1339
Epoch 2/3
1875/1875 ————— 4s 2ms/step - accuracy: 0.8773 - loss: 0.4720
Epoch 3/3
1875/1875 ————— 4s 2ms/step - accuracy: 0.8968 - loss: 0.3737
<keras.src.callbacks.history.History at 0x79a20dff6900>
```

```
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)
```

```
313/313 ————— 0s 1ms/step - accuracy: 0.9063 - loss: 0.3382
Test accuracy: 0.9063000082969666
```

Second Program:

```
... (60000, 28, 28)
(10000, 28, 28)
[9 0 0 ... 3 0 5]
```



```
1875/1875 ————— 4s 2ms/step - accuracy: 0.8248 - loss: 0.5002
Epoch 2/3
1875/1875 ————— 4s 2ms/step - accuracy: 0.8655 - loss: 0.3753
Epoch 3/3
1875/1875 ————— 3s 2ms/step - accuracy: 0.8770 - loss: 0.3393
<keras.src.callbacks.history.History at 0x79a1da5fa420>
```

```
val_loss, val_acc = my_model.evaluate(testX, testy)
print('Test accuracy:', val_acc)
```

```
313/313 ————— 0s 1ms/step - accuracy: 0.8593 - loss: 0.3889
Test accuracy: 0.8593000173568726
```
